Optimization method

Group 1

May 13, 2025

Table of Contents

1. Optimization Algorithm
2. MLP
3. Experimental Process (code structure)
4. Evaluation Indicators and Process
5. Result Analysis

Project Overview and Objectives

Project Introduction

A study on optimization algorithms in the context of machine learning and data science applications, with a focus on:

- Training neural networks using various optimization techniques

Project Goals

- Implement optimization algorithms using the PyTorch framework
- Compare their effectiveness in training neural networks
- Analyze the performance and convergence behavior of each method

Research Motivation

Why Are Optimizers Crucial in Deep Learning?

- Significantly affect training speed and convergence
- Influence final model accuracy and generalization
- Impact computational efficiency and resource utilization

Project Problem and Expected Contribution

- Identify which optimization techniques perform best under different scenarios
- Provide insights into the trade-offs between speed, accuracy, and resource cost
- Contribute practical guidelines for selecting optimizers in real-world applications

What is PyTorch?

Definition

An open-source machine learning framework widely used in:

- Computer Vision (CV)
- Natural Language Processing (NLP)

Core Features

- Dynamic Computational Graphs: Flexible debugging with real-time construction
- Tensors: NumPy-like ndarrays with GPU acceleration
- Autograd System: Automatic differentiation mechanism

PyTorch Core Applications in This Project

torch.nn Module

- Neural network layer construction
- Implementations in `networks.py`:
  - MLP Class (Fully Connected)

Data Loading

- `torch.utils.data`:
  - Dataset
  - DataLoader
- Implementation locations:
  - `misc.py`
  - `main.py`

torch.optim Module

- Optimization algorithms
- Custom extensions in `optimizers.py`

GPU Acceleration

- CUDA(Compute Unified Device Architecture) support
- training speed improvement

Why Choose PyTorch?

Technical Advantages

- Python-first: Seamless ecosystem integration
- Dynamic Graphs: Research-friendly flexibility
- Rich pretrained model library

Project Compatibility

- Custom optimizers:
  - Direct inheritance from `torch.optim.Optimizer`
- Code consistency:
  - `optimizers.py`
  - `networks.py`

Community Support: 65k+ GitHub stars, 92% Stack Overflow resolution rate

CIFAR-10 Benchmark Analysis

Dataset Background

- **Origin**: Curated by Alex Krizhevsky (2009)
- **Structure**:
  - 60,000 32×32 RGB images
  - 50k training / 10k testing
  - 6000 samples per class
- **Applications**:
  - Baseline for MLP architecture validation
  - Benchmark for image recognition systems

MLP Optimization Challenges

- **Low-Resolution** → Spatial feature extraction
- **High Intra-class Variance** → Robust representation learning
- **Color Sensitivity** → Channel-wise processing

CIFAR Dataset Specification

Key Features

- 10 categories of figures

category query

0. Plane
1. Car
2. Bird
3. Cat
4. Deer
5. Dog
6. Frog
7. Horse
8. Ship
9. Truck

Sample



Optimization Algorithm

Optimization Condition

- Require fast convergence (no overshoot at minimum)
- Computation cost is expensive (cannot consider whole dataset)
- Low time complexity $\mathcal{O}$
- Over-fitting is horrible!

- Nesterov Momentum
- RMSProp
- Adam

Nesterov Momentum

Gradient descent will take a long time to traverse a nearly flat surface. Allowing momentum to accumulate is one way to speed progress. We can modify gradient descent to incorporate momentum. The momentum update equations are:

$$\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} - \alpha \mathbf{g}^{(k)} \tag{1}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)} \tag{2}$$

One issue of momentum is that the steps do not slow down enough at the bottom of a valley and tend to overshoot the valley floor.
Nesterov momentum modifies the momentum algorithm to use the gradient at the projected future position:

$$\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)}) \tag{3}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)} \tag{4}$$

RMSProp

RMSProp extends Adagrad to avoid the effect of a monotonically decreasing learning rate. RMSProp maintains a decaying average of squared gradients. This average is updated according to:

$$\hat{\mathbf{s}}^{(k+1)} = \gamma \hat{\mathbf{s}}^{(k)} + (1-\gamma)\left(\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}\right) \tag{5}$$

where the decay $\gamma \in [0, 1]$ is typically close to 0.9.
The decaying average of past squared gradients can be substituted into RMSProp's update equation:

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{\hat{s}_i^{(k)}}} g_i^{(k)} \tag{6}$$

where $\mathbf{s}^{(k)}$ is a vector whose ith entry is the sum of the squares of the partials, with respect to x_i, up to time step k,

$$s_i^{(k)} = \sum_{j=1}^{k} \left(g_i^{(j)}\right)^2 \tag{7}$$

Adam

The adaptive moment estimation method, or Adam, also adapts learning rates to each parameter. It stores both an exponentially decaying squared gradient like *RMSProp* and *Adadelta*, but also an exponentially decaying gradient like momentum.
Initializing the gradient and squared gradient to zero introduces a bias. A bias correction step helps alleviate the issue. The equations applied during each iteration for Adam are:

$$\mathbf{v}^{(k+1)} = \gamma_v \mathbf{v}^{(k)} + (1 - \gamma_v)\, \mathbf{g}^{(k)} \tag{8}$$

$$\mathbf{s}^{(k+1)} = \gamma_s \mathbf{s}^{(k)} + (1 - \gamma_s) \left(\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)} \right) \tag{9}$$

$$\hat{\mathbf{v}}^{(k+1)} = \mathbf{v}^{(k+1)} / \left(1 - \gamma_v^k \right) \tag{10}$$

$$\hat{\mathbf{s}}^{(k+1)} = \mathbf{s}^{(k+1)} / \left(1 - \gamma_s^k \right) \tag{11}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \hat{\mathbf{v}}^{(k+1)} / \left(\epsilon + \sqrt{\hat{s}^{(k+1)}} \right) \tag{12}$$

Introduction to Gradient Dropout

- **Gradient Dropout** is a regularization technique used to prevent overfitting in deep learning models by modifying the gradients during training.
- Instead of applying dropout to the weights (as in standard dropout), gradient dropout is applied to the gradients during backpropagation.
- It is particularly useful in training very deep networks and models that are prone to overfitting.

Benefits

- Helps to reduce overfitting by preventing the model from overly relying on certain features.
- Improves generalization of deep learning models by injecting noise into the gradient computation.
- Useful in architectures that are hard to regularize using traditional dropout methods.

Gradient Noise and Its Role

- Gradient Dropout introduces **gradient noise** into the training process, which acts as a form of regularization.
- **Gradient noise** refers to the random fluctuations in the gradient values caused by the dropout mask applied during backpropagation.
- These fluctuations make the optimization process noisy, preventing the model from overfitting to specific patterns in the data.
- The noise helps avoid the model converging too quickly to sharp minima, which are often associated with overfitting.
- This regularization mechanism is similar to *stochastic gradient descent (SGD)* where the gradients are noisy due to random sampling of the data.

Benefit

- Helps to reduce overfitting by preventing the model from overly relying on certain features.
- Improves generalization of deep learning models by injecting noise into the gradient computation.

Applying Optimizer Knowledge

- Various optimization algorithms (SGD, Adam, etc.).

Applying Optimizer Knowledge

- Various optimization algorithms (SGD, Adam, etc.).
- How algorithms **improve** classification performance.

MLP Structure for CIFAR-10 Classification

Input Layer:

- Flattened CIFAR-10 images: $3 \times 32 \times 32$ (Channels, Height, Width)
- **Flattened** into a vector: $3 \times 32 \times 32 = \mathbf{3072}$ nodes.

Hidden Layer(s):

- Fully connected to input layer.
- Example: 1 or 2 layers with N_h neurons each.
- Activation: Typically ReLU ($\max(0, x)$).

Output Layer:

- Fully connected to the last hidden layer.
- **10 nodes**, one for each CIFAR-10 class (airplane, car, bird, etc.).
- Activation: **Softmax** function to output class probabilities.

$$\text{Softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

Input (3072) $\rightarrow$ [Hidden Layer 1 (N_{h1})] $\rightarrow$ [Hidden Layer 2 (N_{h2})] $\rightarrow$ Output (10)

Forward Pass: From Input to Prediction

How the MLP computes an output $\hat{\mathbf{y}}$ from an input $\mathbf{x}$:

1. **Input to Hidden Layer 1:**
   - Linear step: $\mathbf{z}_{h1} = \mathbf{W}_{h1}\mathbf{x} + \mathbf{b}_{h1}$
   - Activation: $\mathbf{h}_1 = \text{ReLU}(\mathbf{z}_{h1})$

Forward Pass: From Input to Prediction

How the MLP computes an output $\hat{\mathbf{y}}$ from an input $\mathbf{x}$:

1. **Input to Hidden Layer 1:**
   - Linear step: $\mathbf{z}_{h1} = \mathbf{W}_{h1}\mathbf{x} + \mathbf{b}_{h1}$
   - Activation: $\mathbf{h}_1 = \text{ReLU}(\mathbf{z}_{h1})$
2. **Hidden Layer 1 to Hidden Layer 2:**
   - Linear step: $\mathbf{z}_{h2} = \mathbf{W}_{h2}\mathbf{h}_1 + \mathbf{b}_{h2}$
   - Activation: $\mathbf{h}_2 = \text{ReLU}(\mathbf{z}_{h2})$

Forward Pass: From Input to Prediction

How the MLP computes an output $\hat{\mathbf{y}}$ from an input $\mathbf{x}$:

1. **Input to Hidden Layer 1:**
   - Linear step: $\mathbf{z}_{h1} = \mathbf{W}_{h1}\mathbf{x} + \mathbf{b}_{h1}$
   - Activation: $\mathbf{h}_1 = \text{ReLU}(\mathbf{z}_{h1})$
2. **Hidden Layer 1 to Hidden Layer 2:**
   - Linear step: $\mathbf{z}_{h2} = \mathbf{W}_{h2}\mathbf{h}_1 + \mathbf{b}_{h2}$
   - Activation: $\mathbf{h}_2 = \text{ReLU}(\mathbf{z}_{h2})$
3. **Last Hidden Layer to Output Layer:**
   - Linear step: $\mathbf{z}_o = \mathbf{W}_o\mathbf{h}_{\text{last}} + \mathbf{b}_o$

Forward Pass: From Input to Prediction

How the MLP computes an output $\hat{\mathbf{y}}$ from an input $\mathbf{x}$:

1. **Input to Hidden Layer 1:**
   - Linear step: $\mathbf{z}_{h1} = \mathbf{W}_{h1}\mathbf{x} + \mathbf{b}_{h1}$
   - Activation: $\mathbf{h}_1 = \text{ReLU}(\mathbf{z}_{h1})$
2. **Hidden Layer 1 to Hidden Layer 2:**
   - Linear step: $\mathbf{z}_{h2} = \mathbf{W}_{h2}\mathbf{h}_1 + \mathbf{b}_{h2}$
   - Activation: $\mathbf{h}_2 = \text{ReLU}(\mathbf{z}_{h2})$
3. **Last Hidden Layer to Output Layer:**
   - Linear step: $\mathbf{z}_o = \mathbf{W}_o\mathbf{h}_{\text{last}} + \mathbf{b}_o$
4. **Final Prediction (Probabilities):**
   - Activation: $\hat{\mathbf{y}} = \text{Softmax}(\mathbf{z}_o)$

Where:

- $\mathbf{x}$: input vector, $\mathbf{h}$: hidden layer output(s), $\hat{\mathbf{y}}$: predicted probabilities.
- $\mathbf{W}$: weight matrices, $\mathbf{b}$: bias vectors (these are the parameters we learn).
- ReLU, Softmax: activation functions.

Backpropagation: Calculating Gradients

Goal: Find the optimal weights ($\mathbf{W}$) and biases ($\mathbf{b}$) that minimize the difference between predictions ($\hat{\mathbf{y}}$) and true labels ($\mathbf{y}$).

- **Loss Function (L):** Measures the error (e.g., Cross-Entropy Loss for classification).

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{i=1}^{10} y_i \log(\hat{y}_i)$$

Backpropagation: Calculating Gradients

Goal: Find the optimal weights ($\mathbf{W}$) and biases ($\mathbf{b}$) that minimize the difference between predictions ($\hat{\mathbf{y}}$) and true labels ($\mathbf{y}$).

- **Loss Function (L):** Measures the error (e.g., Cross-Entropy Loss for classification).

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{i=1}^{10} y_i \log(\hat{y}_i)$$

- **Backpropagation Algorithm:**
  - Efficiently computes the **gradient** (partial derivative) of the loss L with respect to *every* weight and bias in the network.
  - Gradients: $\nabla_{\mathbf{W}} L = \frac{\partial L}{\partial \mathbf{W}}, \quad \nabla_{\mathbf{b}} L = \frac{\partial L}{\partial \mathbf{b}}$

Optimization: Using Gradients to Update Parameters

Once we have the gradients from backpropagation, how do we update the parameters $(\mathbf{W}, \mathbf{b})$ to *minimize* the loss?

This is where Optimization Algorithms come in!

- **Basic Idea (Gradient Descent):** Move parameters in the opposite direction of the gradient.

$$\mathbf{W}_{\text{new}} = \mathbf{W}_{\text{old}} - \eta \nabla_{\mathbf{W}} L$$

$$\mathbf{b}_{\text{new}} = \mathbf{b}_{\text{old}} - \eta \nabla_{\mathbf{b}} L$$

(η = Learning Rate)

Optimization: Using Gradients to Update Parameters

Once we have the gradients from backpropagation, how do we update the parameters $(\mathbf{W}, \mathbf{b})$ to *minimize* the loss?

This is where Optimization Algorithms come in!

- **Basic Idea (Gradient Descent):** Move parameters in the opposite direction of the gradient.

$$\mathbf{W}_{\text{new}} = \mathbf{W}_{\text{old}} - \eta \nabla_{\mathbf{W}} L$$

$$\mathbf{b}_{\text{new}} = \mathbf{b}_{\text{old}} - \eta \nabla_{\mathbf{b}} L$$

  (η = Learning Rate)
- **Challenges with Simple Gradient Descent:**
  - Sensitivity to learning rate η.
  - Can get stuck in local minima or saddle points.
  - Slow convergence in certain "ravine" landscapes.
  - Doesn't adapt step size per parameter.

Enhancing Updates: Role of Advanced Optimizers

Optimizers use the gradients (∇L) from backpropagation more intelligently:

- **SGD with Momentum / Nesterov:**
  - Adds inertia (momentum) to the update step.
  - Helps overcome small local minima, accelerates convergence in consistent directions, dampens oscillations.
  - Nesterov looks ahead for better gradient calculation.

Enhancing Updates: Role of Advanced Optimizers

Optimizers use the gradients (∇L) from backpropagation more intelligently:

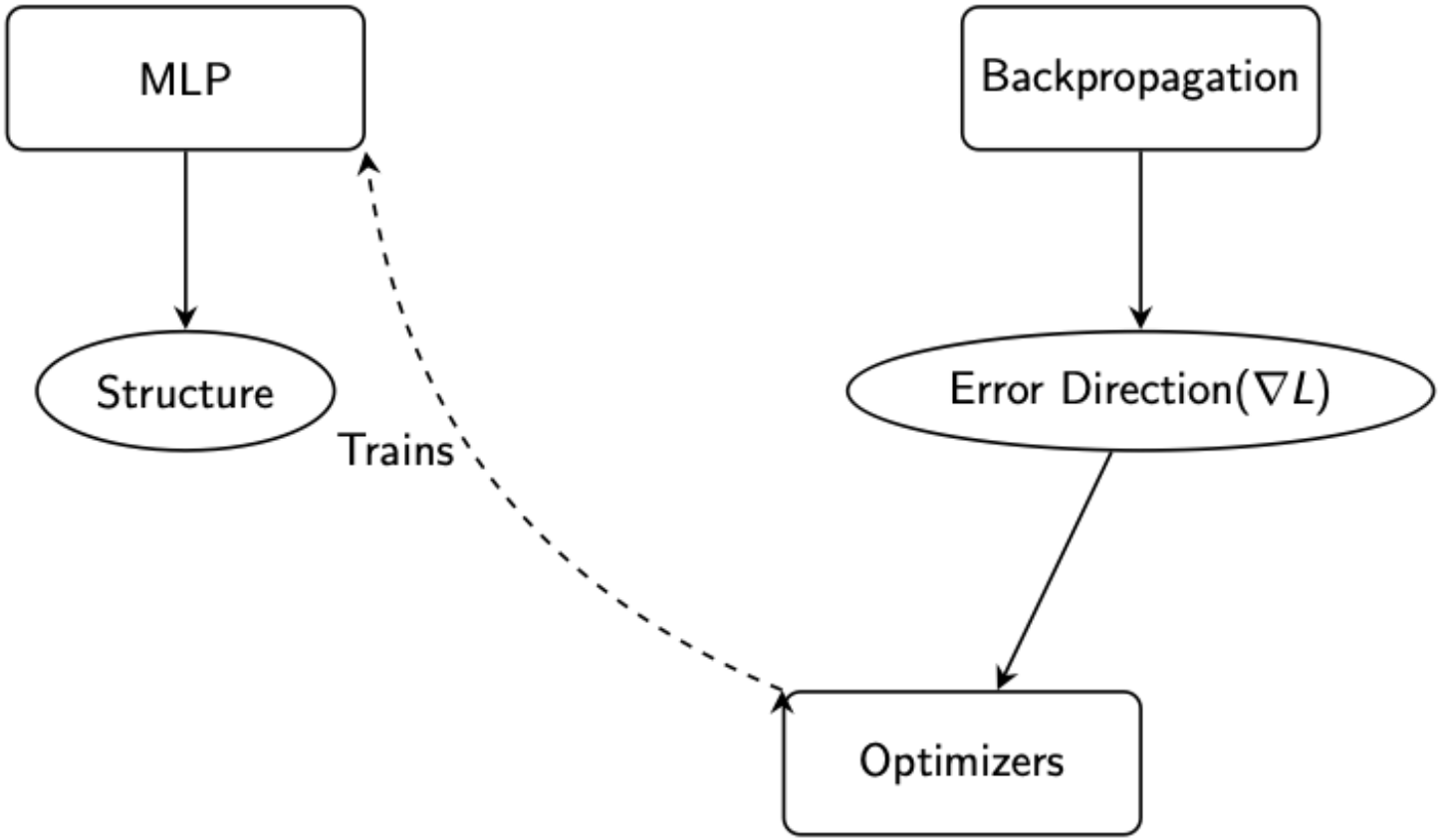
- **SGD with Momentum / Nesterov:**
  - Adds inertia (momentum) to the update step.
  - Helps overcome small local minima, accelerates convergence in consistent directions, dampens oscillations.
  - Nesterov looks ahead for better gradient calculation.
- **Adam / Nadam / RAdam:**
  - **Adaptive Learning Rates:** Computes individual learning rates for each parameter based on estimates of gradient moments (mean and uncentered variance).
  - Often converges faster and is less sensitive to the initial learning rate.
  - Nadam incorporates Nesterov momentum. RAdam adds rectification for better stability early in training.

Enhancing Updates: Role of Advanced Optimizers

Optimizers use the gradients (∇L) from backpropagation more intelligently:

- **SGD with Momentum / Nesterov:**
  - Adds inertia (momentum) to the update step.
  - Helps overcome small local minima, accelerates convergence in consistent directions, dampens oscillations.
  - Nesterov looks ahead for better gradient calculation.
- **Adam / Nadam / RAdam:**
  - **Adaptive Learning Rates:** Computes individual learning rates for each parameter based on estimates of gradient moments (mean and uncentered variance).
  - Often converges faster and is less sensitive to the initial learning rate.
  - Nadam incorporates Nesterov momentum. RAdam adds rectification for better stability early in training.
- **RMSProp:**
  - Also uses adaptive learning rates, based on a moving average of squared gradients.
  - Effective at handling non-stationary objectives. (Foundation for Adam).

Loop Structure



Experimental Process (code structure)

1. Code Modules
   - **Main.py**: Coordinates the full experiment workflow
   - **Networks.py**: Defines neural network models
   - **Optimizers.py**: Implements optimization algorithms
   - **Misc.py**: Provides auxiliary functionalities
2. Experimental Workflow

1. Code Modules

Networks.py

- **Neural Network Model**
  - Multilayer Perceptron (MLP)
  - Fully connected layers
  - Flattened CIFAR-10 input: $3 \times 32 \times 32 \rightarrow$ 3072-D
- **Training Function (`fit()`)**
  - Runs train/test loops
  - Logs epoch loss and accuracy
  - Returns history: {loss, acc (train/val)}

1. Code Modules

Optimizers.py

Using code to implement several optimization algorithms.

- **Optimization Algorithms**
  - SGD
  - Adam
  - RMSProp
  - L-BFGS
- **Optimizer Wrappers**
  - `Lookahead`: Stabilizes training
  - `GradientNoise`: Adds Gaussian noise to escape local minima
  - `GradientDropout`: Drops gradient components for improved generalization

1. Code Modules

Misc.py

Provides auxiliary functionalities, including data loading, optimizer configuration, result processing, and visualization.

- **Data Loading**
  - `load_cifar`: Loads CIFAR-10 dataset
- **Optimizer Configuration**
  - `optim_dict`: Predefined configurations for optimizers (SGD, Adam, L-BFGS, etc.)
  - `get_optimizer`: Generates optimizer instances
  - `get_mixer_configs`: Expands `optim_dict` for detailed configurations
- **Result Processing**
  - `AvgMetric`: Tracks loss and accuracy per epoch
  - `save/load_metric_history`: Serializes/deserializes metrics
- **Visualization**
  - `plot_all_metrics`: Generates comparative plots (loss, accuracy)

2. Experimental Workflow

Command

```
!python main.py -dataset cifar -num_epochs 30 -num_inits 5
```

1. **Initialization**: Set random seeds, select CPU/GPU
2. **Data Loading**: Load CIFAR-10 via `load_cifar` (`misc.py`)
3. **Optimizer Configuration**: Retrieve optimizers from `optim_dict`
4. **Training Phase**
   - Run each optimizer `num_inits` times
   - Train using `fit()` (`networks.py`)
5. **Result Aggregation**: Compute mean $\pm$ std, save to `.pkl`
6. **Visualization**: Plot loss, accuracy, efficiency curves

Evaluation Indicators and Process

1. Evaluation Indicators
2. Evaluation Process

1. Evaluation Indicators

- **Train/Test Loss**
  - Cross-Entropy Loss to measure divergence between predictions and truth
- **Train/Test Accuracy**
  - Percentage of correctly predicted samples
- **Computational Efficiency**
  - Tracks running time (`run_elapsed_time`) per optimizer

2. Evaluation Process

1. **Train**: Compute train loss and accuracy per epoch
2. **Test**: Compute test loss and accuracy per epoch
3. **Log**: Use `AvgMetric` to log epoch metrics
4. **Aggregate**: Compute mean $\pm$ std across initializations
5. **Compare**: Rank optimizers by final validation accuracy and generate a summary table

Comparison of Optimizers (1/2)

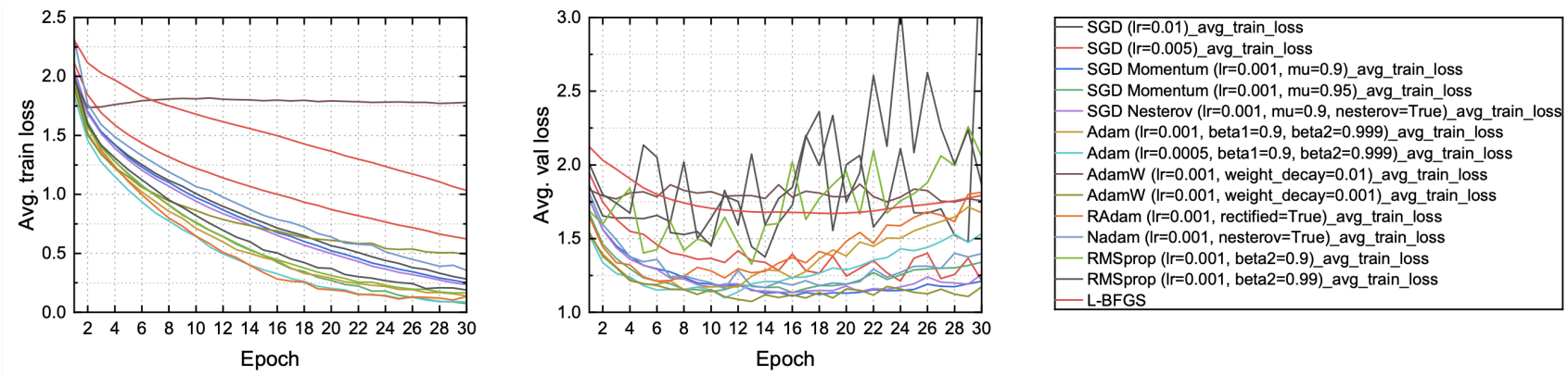


Figure: Epoch Visualization

Comparison of Optimizers (2/2)

Full_Optimizer_Comparison_Summary

Optimizer	Final Val Acc	Final Val Loss	Avg Train Time (s)
SGD Momentum (0.001, 0.95)	0.6276	1.3399	73.57
SGD (lr=0.01)	0.4118	3.651807707	73.65662684
SGD (lr=0.005)	0.5756	1.214174392	73.51772318
SGD Momentum (lr=0.001, mu=0.9)	0.6052	1.210454307	73.43155899
SGD Nesterov (lr=0.001, mu=0.9, nesterov=True)	0.5974	1.257890946	75.97145061
Adam (0.0005)	0.6242	1.5315	78.91
AdamW (0.001, wd=0.001)	0.5952	1.1717	78.86
l-bfgs	0.41015625	1.824126959	43.96463752
Nadam (lr=0.001, nesterov=True)	0.6106	1.398217904	77.90338211
RAdam (lr=0.001, rectified=True)	0.6038	1.814655958	79.33495307
RMSprop (lr=0.001, beta2=0.9)	0.598	2.055189225	78.20587597
RMSprop (lr=0.001, beta2=0.99)	0.6028	1.865687139	78.44445753

Figure: Data Visualization

Future Work

Potential Extensions

- Explore more advanced optimizers or optimizer wrappers
- Apply techniques to different neural network architectures or datasets
- Conduct deeper research into hyperparameter optimization